

PLACEMENT REVISION SERIES

DBMS Schema, Types & Three-Schema Architecture

Quick reference notes for core Database Management Systems interview topics

1. Core Concepts: Schema vs. Instance

In DBMS, database design and the database itself are treated as separate entities. Understanding the difference between a schema and an instance is foundational for core technical interviews.

DATABASE SCHEMA (INTENSION)

The **Schema** is the logical structural definition of the database. It outlines how data is organized, including tables (relations), attributes (columns), data types, constraints, and relationships. It is designed during database creation, is highly static, and rarely changes.

Programming Metaphor: A Schema is equivalent to a **Class** in Object-Oriented Programming (OOP).

DATABASE INSTANCE (EXTENSION)

The **Instance** is the actual data stored in the database at a specific moment in time. Whenever data is inserted, updated, or deleted, the instance changes immediately. It is dynamic and representation-heavy.

Programming Metaphor: An Instance is equivalent to an ****Object**** (an instantiated variable of a class) in OOP.

Key Parameter	Database Schema	Database Instance
Definition	Logical outline/structure of the database.	Actual data values stored in the database at any moment.

Key Parameter	Database Schema	Database Instance
Frequency of Change	Extremely rare (requires schema migration or DDL execution).	Very high (changes with every write operation/DML execution).
Formal Name	Intension	Extension
SQL Language Used	DDL (Data Definition Language) - e.g., <code>CREATE</code> , <code>ALTER</code>	DML (Data Manipulation Language) - e.g., <code>INSERT</code> , <code>UPDATE</code>

2. Types of DBMS Schema

Database architectures categorize schemas based on their proximity to physical storage versus end-users. There are three types of schemas:

- **Physical Schema:** Defines how data is physically stored in storage media (HDDs/SSDs) as files, blocks, clusters, and indexes. It represents the database at the lowest level of abstraction.
- **Logical Schema:** Defines the tables, views, integrity constraints, and relationships between tables. It describes what data is stored in the database in a structural sense, hiding physical storage details.
- **View Schema (or Subschema):** Defines customized subsets of the logical schema targeted at specific end-users or application modules. It describes only the relevant portion of the database for that user group.

3. The Three-Schema Architecture (ANSI-SPARC)

The **Three-Schema Architecture**, proposed by the ANSI-SPARC Committee in 1975, is a conceptual framework designed to decouple user applications from physical database structures. The main goal of this architecture is to achieve **Data Independence**.

It segregates the database into three levels of abstraction, linked by translation mechanisms called **mappings**.

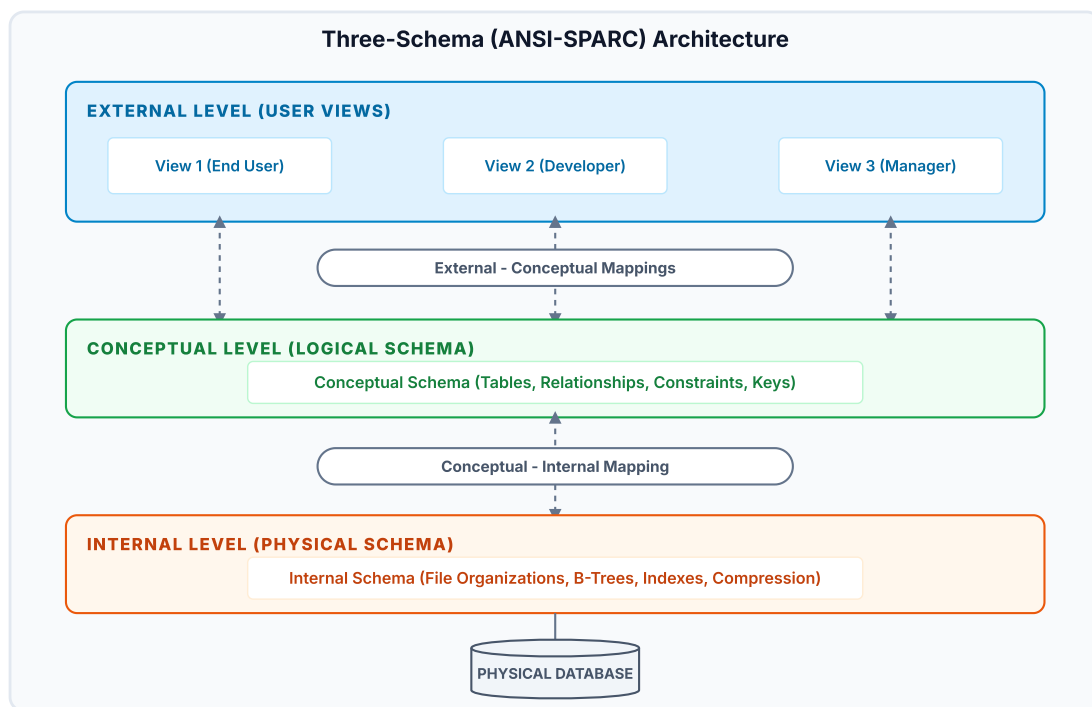


Figure 1: The ANSI-SPARC 3-Schema Architecture with View, Logical, and Physical abstractions.

The Three Abstraction Levels

- 1. External Level (View Level):** The highest level of data abstraction. It represents views customized to different user categories. Users only interact with their specific view and remain unaware of the rest of the database structures.

2. **Conceptual Level (Logical Level):** The intermediate level. It describes **what** data is stored and the relationships among them. It represents the global blueprint of the database. It contains table schemas, logical constraints (e.g., uniqueness), and relationships (foreign keys). It has zero physical details.
3. **Internal Level (Physical Level):** The lowest level. It describes **how** the data is physically stored on disk. It handles page sizes, byte offsets, indexing paths, B-Tree allocations, hashing details, and compression schemes.

4. Database Mappings

Database mappings are the translation pathways between adjacent levels. When an application queries an external view, the DBMS uses mappings to translate the query into operations on logical tables, and then into physical files.

- **External-Conceptual Mapping:** Translates requests from a specific user view (External Level) to the global logical schema (Conceptual Level). If you modify an external view, this mapping must be updated.
- **Conceptual-Internal Mapping:** Translates requests from the logical structure (Conceptual Level) to the physical structure (Internal Level). It maps logical tables to actual records, files, indexes, and directories on disk.

5. Data Independence

Data Independence is the ability to modify the schema at one level without having to alter the schema at the level above it. This prevents database changes from breaking client applications.

Physical Data Independence	Logical Data Independence
Allows changing the Internal/Physical Schema without having to change the Conceptual/Logical Schema or user programs.	Allows changing the Conceptual/Logical Schema without having to change the External Schemas or user programs.
Changes are restricted to physical storage details (e.g., adding an index, changing storage device type, altering file systems, using hashing instead of B-trees).	Changes are restricted to structural rules (e.g., adding/deleting attributes to a table, splitting one table into two, merging two tables).
Easy to achieve because the mapping rules from logical structure to physical files are relatively straightforward.	Hard to achieve because external views depend heavily on the logical structures, and splitting tables requires complex updates to mapping layers.

INTERVIEW TIP: WHY IS LOGICAL DATA INDEPENDENCE HARDER TO ACHIEVE?

If you add or remove columns, or split a table into two (Normalization), any client application running standard SQL SELECT queries directly on the original table will fail unless you write a View that mimics the old table structure. Creating and maintaining these views and their mappings makes Logical Data Independence much more difficult to support compared to Physical Data Independence (where the query syntax stays identical regardless of index changes).

6. Placement Interview Q&A

Q1. What is the fundamental difference between Schema and Instance?

The **Schema** is the overall structure or design of the database, defined by metadata (DDL) and is highly static. The **Instance** is the snapshot of actual data stored in the database at a specific timestamp, changed by DML queries, and is highly dynamic.

Q2. Why do we need the Three-Schema Architecture?

It is needed to decouple the physical storage structure from user-facing applications. This ensures that physical improvements (such as adding indexes or changing hardware) or logical restructurings (such as normalizations) do not break the front-end application code, thereby achieving Data Independence.

Q3. What is Metadata and where is it stored?

Metadata is "data about data"—information that describes the schema structure, constraints, indexes, and table definitions. In a DBMS, it is stored in the **Data Dictionary** (also known as the System Catalog), which itself is a set of read-only system tables.

Q4. Can you give a practical example of Physical Data Independence?

Suppose we have a table `Employees`. If we execute a query like `SELECT * FROM Employees WHERE EmpID = 101`, and then we decide to create an index on `EmpID` to optimize performance, the SQL query does not change. The logical view of the table is untouched; only the internal traversal path changes. This is physical data independence.

Q5. Who defines the Conceptual Schema?

The **Database Administrator (DBA)** and database designers define the Conceptual Schema. They identify entities, attributes, primary/foreign keys, and validation checks

during the design phase of the database development life cycle.